

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum, Karnataka -590014.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

P Nithya (1BF24CS206)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bengaluru 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by P Nithya (1BF24CS206), who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	4-15
2.	Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	16-19
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	20-24
4	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem.	25-30
5	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	31-38
6	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	39-44
7	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	45-50

Lab 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n,i;
    int at[10],bt[10],ct[10],tat[10],wt[10];
    printf("\nEnter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\nEnter Arrival time and Burst time for P%d ",i+1);
        scanf("%d %d",&at[i],&bt[i]);
    }
    ct[0] = at[0] + bt[0];
    for(i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
    for(i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    return 0;
}
```

OUTPUT:

```
C:\Users\student\Documents\ X + v
Enter number of processes: 4
Enter Arrival Time:
P1: 0
P2: 2
P3: 4
P4: 5
Enter Burst Time:
P1: 7
P2: 4
P3: 1
P4: 4

Process AT BT CT WT TAT RT
P1      0 7 7 0 7 0
P2      2 4 11 5 9 5
P3      4 1 12 7 8 7
P4      5 4 16 7 11 7

Average Waiting Time = 4.75
Average Turnaround Time = 8.75
Average Response Time = 4.75

Process returned 0 (0x0)  execution time : 31.060 s
Press any key to continue.
|
```

b) SJF : Non Pre-emptive and Pre-emptive

```
#include <stdio.h>

#include <limits.h>

int main() {

    int n,i,time=0,done=0,choice;

    int at[10],bt[10],rt[10],ct[10],wt[10],tat[10];

    printf("Enter number of processes: ");

    scanf("%d",&n);

    for(i=0;i<n;i++){

        printf("AT and BT of P%d: ",i+1);

        scanf("%d%d",&at[i],&bt[i]);

        rt[i]=bt[i];

    }

    printf("\nSelect Type:\n");

    printf("1. Non-Preemptive\n");

    printf("2. Preemptive\n");

    printf("Enter choice: ");

    scanf("%d", &choice);

    if(choice==1){ // Non-preemptive

        int vis[10]={0};

        while(done<n){

            int min=INT_MAX,p=-1;
```

```

for(i=0;i<n;i++)

    if(at[i]<=time && !vis[i] && bt[i]<min){

        min=bt[i]; p=i;

    }

if(p!=-1){

    time++;

    continue;

}

time+=bt[p];

ct[p]=time;

vis[p]=1;

done++;

}

}

else{ // Preemptive (SRTF)

while(done<n){

    int min=INT_MAX,p=-1;

    for(i=0;i<n;i++)

        if(at[i]<=time && rt[i]>0 && rt[i]<min){

            min=rt[i]; p=i;

        }

    if(p!=-1){ time++; continue; }

    rt[p]--; time++;

```

```

        if(rt[p]==0){

            ct[p]=time;

            done++;

        }

    }

}

float tsum=0;

float wsum=0;

printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++){

    tat[i]=ct[i]-at[i];

    wt[i]=tat[i]-bt[i];

    tsum+=tat[i];

    wsum+=wt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],ct[i],tat[i],wt[i]);

}

printf("Average TAT:%f, WT:%f",tsum,wsum);

return 0;

}

```

OUTPUT:

```

C:\Users\student\Documents\ X + v
Enter number of processes: 4
AT and BT of P1: 0 2
AT and BT of P2: 1 3
AT and BT of P3: 3 1
AT and BT of P4: 4 5

Select Type:
1. Non-Preemptive
2. Preemptive
Enter choice: 2

P      AT      BT      CT      TAT      WT
P1      0      2      2      2      0
P2      1      3      6      5      2
P3      3      1      4      1      0
P4      4      5      11     7      2
Average TAT:15.000000, WT:4.000000
Process returned 0 (0x0)  execution time : 15.271 s
Press any key to continue.
|

```

```

C:\Users\student\Documents\ X + v
Enter number of processes: 4
AT and BT of P1: 0 2
AT and BT of P2: 2 3
AT and BT of P3: 1 4
AT and BT of P4: 3 1

Select Type:
1. Non-Preemptive
2. Preemptive
Enter choice: 1

P      AT      BT      CT      TAT      WT
P1      0      2      2      2      0
P2      2      3      5      3      0
P3      1      4      10     9      5
P4      3      1      6      3      2
Average TAT:17.000000, WT:7.000000
Process returned 0 (0x0)  execution time : 14.911 s
Press any key to continue.
|

```

c) Priority Non-Pre-emptive and Pre-emptive:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int n,i,choice;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int at[n],bt[n],rt[n],pr[n];
    int ct[n],wt[n],tat[n];

    for(i=0;i<n;i++){
        printf("Enter AT, BT and Priority of P%d: ",i+1);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
        rt[i]=bt[i];
    }

    printf("\n1. Non Preemptive Priority\n");
    printf("2. Preemptive Priority\n");
    printf("Enter choice: ");
    scanf("%d",&choice);
    int time=0,count=0;

    if(choice==1){
        int completed[n];
        for(i=0;i<n;i++) completed[i]=0;

        while(count<n){
            int idx=-1;
            int highest=9999;

            for(i=0;i<n;i++){
                if(at[i]<=time && completed[i]==0){
                    if(pr[i]<highest){
                        highest=pr[i];
                        idx=i;
                    }
                }
            }

            if(idx!=-1){
                time+=bt[idx];

                ct[idx]=time;
                tat[idx]=ct[idx]-at[idx];
                wt[idx]=tat[idx]-bt[idx];

                completed[idx]=1;
            }
        }
    }
}
```

```

        count++;
    }
    else{
        time++;
    }
}
}

else if(choice==2){
    while(count<n){
        int idx=-1;
        int highest=9999;

        for(i=0;i<n;i++){
            if(at[i]<=time && rt[i]>0){
                if(pr[i]<highest){
                    highest=pr[i];
                    idx=i;
                }
            }
        }

        if(idx!=-1){
            rt[idx]--;
            time++;

            if(rt[idx]==0){
                ct[idx]=time;
                tat[idx]=ct[idx]-at[idx];
                wt[idx]=tat[idx]-bt[idx];
                count++;
            }
        }
        else{
            time++;
        }
    }
}

printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
}

return 0;

```

OUTPUT:

```

C:\Users\student\Documents\ X + v
Enter number of processes: 3
Enter AT, BT and Priority of P1: 0 2 1
Enter AT, BT and Priority of P2: 0 6 3
Enter AT, BT and Priority of P3: 4 2 2

1. Non Preemptive Priority
2. Preemptive Priority
Enter choice: 2

P      AT      BT      PR      CT      TAT      WT
P1      0        2        1        2        2        0
P2      0        6        3       10       10        4
P3      4        2        2        6        2        0
Average TAT:4.666667, WT:1.333333
Process returned 0 (0x0)  execution time : 47.745 s
Press any key to continue.
|

```

```

C:\Users\student\Documents\ X + v
Enter number of processes: 3
Enter AT, BT and Priority of P1: 0 2 1
Enter AT, BT and Priority of P2: 0 6 3
Enter AT, BT and Priority of P3: 4 2 2

1. Non Preemptive Priority
2. Preemptive Priority
Enter choice: 1

P      AT      BT      PR      CT      TAT      WT
P1      0        2        1        2        2        0
P2      0        6        3        8        8        2
P3      4        2        2       10        6        4
Average TAT:5.333333, WT:2.000000
Process returned 0 (0x0)  execution time : 22.149 s
Press any key to continue.
|

```

d) Round robin

```

#include<stdio.h> struct
process{
    int pid, at, bt, ct, tat, wt, rt;
};

int main(){
    int n, i, time = 0, remain, tq, done;
    float avg_wt=0, avg_tat=0;
    struct process p[20];
    printf("enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        p[i].pid = i+1;
        printf("enter at and bt for P%d: ",p[i].pid);
        scanf("%d %d",&p[i].at,&p[i].bt);
        p[i].rt = p[i].bt;
    }
    printf("enter time quantum: ");
    scanf("%d",&tq);
    remain = n;
    while(remain != 0){
        done = 1;
        for(i = 0; i < n; i++){
            if(p[i].rt > 0 && p[i].at <= time){ done
                = 0;
                if(p[i].rt > tq){ time
                    += tq; p[i].rt -= tq;
                }
            }
            else{

```

```

        time += p[i].rt; p[i].rt
        = 0;
        p[i].ct = time;
        remain--;
    }
}
}
if(done == 1){
    time++;
}
}
for(i = 0; i < n; i++){ p[i].tat =
    p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg_wt += p[i].wt; avg_tat
    += p[i].tat;
}
printf("\npid\tat\tbt\tct\ttat\twt\n");
for(i = 0; i < n; i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}
printf("\naverage tat = %.2f", avg_tat/n);
printf("\naverage wt = %.2f\n", avg_wt/n);

return 0;
}

```

OUTPUT:

```

enter number of processes: 4
enter at and bt for P1: 0
7
enter at and bt for P2: 2
4
enter at and bt for P3: 4
9
enter at and bt for P4: 5
5
enter time quantum: 2

```

pid	at	bt	ct	tat	wt
P1	0	7	22	22	15
P2	2	4	12	10	6
P3	4	9	25	21	12
P4	5	5	21	16	11

```

average tat = 17.25
average wt  = 11.00

```

```

enter number of processes: 4
enter at and bt for P1: 0
7
enter at and bt for P2: 2
4
enter at and bt for P3: 4
9
enter at and bt for P4: 5
5
enter time quantum: 4

```

pid	at	bt	ct	tat	wt
P1	0	7	19	19	12
P2	2	4	8	6	2
P3	4	9	25	21	12
P4	5	5	24	19	14

```

average tat = 16.25
average wt  = 10.00

```

```

Process returned 0 (0x0)   execution time : 30.179 s
Press any key to continue.

```

LAB 2

Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include<stdio.h>

struct process{
    int pid, at, bt, ct, tat, wt, rt; int
    type;
};

int main(){
    int n, i, time = 0, remain, done; float
    avg_wt = 0, avg_tat = 0; struct
    process p[20];
    printf("Enter number of processes: "); scanf("%d",
    &n);
    for(i = 0; i < n; i++){
        p[i].pid = i + 1;
        printf("Enter arrival time, burst time, and type (0 = System / 1 = User) for P%d: ", p[i].pid);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].type); p[i].rt =
        p[i].bt;
    }
    remain = n;
    while(remain != 0){
        done = 1;
        for(i = 0; i < n; i++){
            if(p[i].rt > 0 && p[i].at <= time && p[i].type == 0){
                done = 0;
                time++;
            }
        }
        if(done == 1) break;
        remain--;
    }
}
```

```

    p[i].rt--;
    if(p[i].rt == 0){ p[i].ct =
        time;
        remain--;
    }
    break;
}
}
if(done == 1){
    for(i = 0; i < n; i++){
        if(p[i].rt > 0 && p[i].at <= time && p[i].type == 1){ done =
            0;
            time++;
            p[i].rt--;
            if(p[i].rt == 0){
                p[i].ct = time;
                remain--;
            }
            break;
        }
    }
}
if(done == 1){
    time++;
}
}
for(i = 0; i < n; i++){ p[i].tat =
    p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg_wt += p[i].wt; avg_tat
    += p[i].tat;
}

```

```
}  
printf("\nPID\tType\tAT\tBT\tCT\tTAT\tWT\n");  
for(i = 0; i < n; i++){  
    printf("P%d\t%s\t%d\t%d\t%d\t%d\t%d\n",  
        p[i].pid,  
        (p[i].type == 0) ? "System" : "User", p[i].at,  
        p[i].bt, p[i].ct, p[i].tat, p[i].wt);  
}  
printf("\nAverage Turnaround Time: %.2f", avg_tat/n);  
printf("\nAverage Waiting Time: %.2f\n", avg_wt/n); return 0;  
}
```

OUTPUT:

```

C:\Users\BMSCECSE\Desktop\ X + v
Enter number of processes: 4

Process 1
Enter arrival time: 0
Enter burst time: 3
Enter type (0 = System, 1 = User): 0

Process 2
Enter arrival time: 2
Enter burst time: 5
Enter type (0 = System, 1 = User): 1

Process 3
Enter arrival time: 3
Enter burst time: 7
Enter type (0 = System, 1 = User): 0

Process 4
Enter arrival time: 5
Enter burst time: 3
Enter type (0 = System, 1 = User): 0

ID      Type    AT    BT    CT    WT    TAT
1       System  0      3     3     0     3
2       User    2      5    18    11    16
3       System  3      7    10     0     7
4       System  5      3    13     5     8

Average Waiting Time: 4.00
Average Turnaround Time: 8.50

Process returned 0 (0x0)  execution time : 34.974 s
Press any key to continue.

```

LAB 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MAX 10 typedef
struct {
    int id, burst, deadline, period, remaining, weight;
} Process;

void sortEDF(Process p[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=i+1;j<n;j++) { if(p[i].deadline
            > p[j].deadline) {
                Process temp = p[i]; p[i]
                = p[j];
                p[j] = temp;
            }
        }
    }
}

void sortRMS(Process p[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=i+1;j<n;j++) {
            if(p[i].period > p[j].period) {
                Process temp = p[i]; p[i]
                = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```

    }
}

void EDF(Process p[], int n) {
    float utilization = 0;
    for(int i=0;i<n;i++)
        utilization += (float)p[i].burst / p[i].deadline;
    sortEDF(p, n);
    printf("\n===== Earliest Deadline First (EDF) =====\n");
    printf("CPU Utilization: %.2f\n", utilization); printf("Schedulable:
%s\n", (utilization <= 1.0) ? "YES" : "NO"); int time = 0;
    printf("ID\tBT\tDL\tCT\tWT\tTAT\n");
    for(int i=0;i<n;i++) {
        int ct = time + p[i].burst; int
        tat = ct;
        int wt = tat - p[i].burst;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].burst, p[i].deadline, ct, wt, tat); time
        = ct;
    }
}

void RMS(Process p[], int n) {
    float utilization = 0;
    for(int i=0;i<n;i++)
        utilization += (float)p[i].burst / p[i].period; float
    bound = n * (pow(2, 1.0/n) - 1); sortRMS(p, n);
    printf("\n===== Rate Monotonic Scheduling (RMS) =====\n");
    printf("CPU Utilization: %.2f\n", utilization);
    printf("RM Bound: %.4f\n", bound);
    printf("Schedulable: %s\n", (utilization <= bound) ? "YES" : "NO");
}

```

```

int time = 0;
printf("ID\tBT\tPR\tCT\tWT\tTAT\n");
for(int i=0;i<n;i++) {
    int ct = time + p[i].burst; int
    tat = ct;
    int wt = tat - p[i].burst;
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id, p[i].burst, p[i].period, ct, wt, tat);
    time = ct;
}
}

void Proportional(Process p[], int n) { int
    total_weight = 0;
    for(int i=0;i<n;i++) total_weight
        += p[i].weight;
    printf("\n===== Proportional Scheduling =====\n");
    printf("ID\tWeight\tCPU Share(%%)\n");
    for(int i=0;i<n;i++) {
        float share = ((float)p[i].weight / total_weight) * 100;
        printf("%d\t%d\t%.2f%%\n", p[i].id, p[i].weight, share);
    }
}

int main() { int
    n;
    Process p[MAX];
    printf("Enter number of processes: "); scanf("%d",
    &n);
    for(int i=0;i<n;i++) {
        p[i].id = i;
        printf("\nProcess %d\n", i);
        printf("Burst Time: ");

```

```

scanf("%d", &p[i].burst);
printf("Deadline: ");
scanf("%d", &p[i].deadline);
printf("Period: "); scanf("%d",
&p[i].period);
printf("Weight (for proportional scheduling): ");
scanf("%d", &p[i].weight);
}
EDF(p, n);
RMS(p, n);
Proportional(p, n);
return 0;
}

```

OUTPUT:

```

Enter number of processes: 3
Enter process details:
Process 0:
Burst Time: 2
Deadline (for EDF): 5
Period (for RMS): 5
Process 1:
Burst Time: 2
Deadline (for EDF): 7
Period (for RMS): 7
Process 2:
Burst Time: 3
Deadline (for EDF): 8
Period (for RMS): 8
===== Earliest Deadline First (EDF) Scheduling =====
CPU Utilization: 1.06


| ID | BF | Deadline | CT | WT | TAT |
|----|----|----------|----|----|-----|
| 0  | 2  | 5        | 2  | 0  | 2   |
| 1  | 2  | 7        | 4  | 2  | 4   |
| 2  | 3  | 8        | 7  | 4  | 7   |


===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 1.06
RM Bound: 0.7798


| ID | BF | Period | CT | WT | TAT |
|----|----|--------|----|----|-----|
| 0  | 2  | 5      | 2  | 0  | 2   |
| 1  | 2  | 7      | 4  | 2  | 4   |
| 2  | 3  | 8      | 7  | 4  | 7   |


```

```

"C:\Users\maany\OneDrive\D  × + ∨

Process 0
Burst Time: 3
Deadline: 7
Period: 20
Weight (for proportional scheduling): 4

Process 1
Burst Time: 2
Deadline: 4
Period: 5
Weight (for proportional scheduling): 2

Process 2
Burst Time: 2
Deadline: 8
Period: 10
Weight (for proportional scheduling): 4

===== Earliest Deadline First (EDF) =====
CPU Utilization: 1.18
Schedulable: NO
ID      BT      DL      CT      WT      TAT
1        2        4        2        0        2
0        3        7        5        2        5
2        2        8        7        5        7

===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 0.75
RM Bound: 0.7798
Schedulable: YES
ID      BT      PR      CT      WT      TAT
1        2        5        2        0        2
2        2       10        4        2        4
0        3       20        7        4        7

===== Proportional Scheduling =====
ID      Weight  CPU Share(%)
1        2      20.00%
2        4      40.00%
0        4      40.00%

```

LAB 4

Write a C program to simulate: a) Producer-Consumer problem using semaphores.
b) Dining-Philosopher's problem

a) Producer-Consumer problem

```
#include <stdio.h> #include
<stdlib.h> #include
<pthread.h> #include
<semaphore.h> #include
<unistd.h> #define
BUFFER_SIZE 5
#define MAX 15

int buffer[BUFFER_SIZE]; int
in = 0, out = 0;
sem_t empty, full, mutex;

void* producer(void* arg) {
    for (int i = 0; i < MAX; i++) {
        sem_wait(&empty);
        sem_wait(&mutex); buffer[in]
        = i;
        printf("Produced: %d at buffer[%d]\n", i, in); in =
        (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        sleep(1);
    }
    return NULL;
}
```

```

void* consumer(void* arg) {
    for (int i = 0; i < MAX; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out); out
        = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        sleep(1);
    }
    return NULL;
}

```

```

int main() { pthread_t
    p, c;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex); return
    0;
}

```

OUTPUT:

```
"C:\Users\maany\OneDrive\  × + v
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Consumed: 2 from buffer[2]
Produced: 3 at buffer[3]
Consumed: 3 from buffer[3]
Produced: 4 at buffer[4]
Consumed: 4 from buffer[4]
Produced: 5 at buffer[0]
Consumed: 5 from buffer[0]
Produced: 6 at buffer[1]
Consumed: 6 from buffer[1]
Produced: 7 at buffer[2]
Consumed: 7 from buffer[2]
Produced: 8 at buffer[3]
Consumed: 8 from buffer[3]
Produced: 9 at buffer[4]
Consumed: 9 from buffer[4]
Produced: 10 at buffer[0]
Consumed: 10 from buffer[0]
Produced: 11 at buffer[1]
Consumed: 11 from buffer[1]
Produced: 12 at buffer[2]
Consumed: 12 from buffer[2]
Produced: 13 at buffer[3]
Consumed: 13 from buffer[3]
Produced: 14 at buffer[4]
Consumed: 14 from buffer[4]

Process returned 0 (0x0)   execution time : 15.973 s
Press any key to continue.
|
```

b) Dining-Philosopher's problem

```
#include <stdio.h> #include
<pthread.h> #include
<semaphore.h> #include
<unistd.h> #define N 5

sem_t forks[N];
sem_t room;

void* philosopher(void* num) { int
    id = *(int*)num;
    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        sem_wait(&room); if
        (id % 2 == 0) {
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left fork %d.\n", id, id);
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);
        } else {
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left fork %d.\n", id, id);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(1);
        sem_post(&forks[id]);
        sem_post(&forks[(id + 1) % N]);
    }
}
```

```

        printf("Philosopher %d put down forks %d and %d.\n", id, id, (id + 1) % N);
        sem_post(&room);
    }
}

int main() { pthread_t
    phil[N]; int ids[N];
    sem_init(&room, 0, N - 1); for
    (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }
    for (int i = 0; i < N; i++) { ids[i]
        = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(phil[i], NULL);
    }
    return 0;
}

```

OUTPUT:

```
"C:\Users\maany\OneDrive\D  × + v
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 0 put down forks 0 and 1.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 is thinking.
```

LAB 5

Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection

a) Bankers' algorithm

```
#include <stdio.h>

int main() {
    int n,m;
    int i, j, k;
    printf("Enter n and m values: ");
    scanf("%d %d",&n,&m);
    printf("\nEnter Allocation matrix: ");
    int alloc[n][m];
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("\n%d",&alloc[i][j]);
        }
    }
    printf("\nEnter Maximum matrix: ");
    int max[n][m];
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("\n%d",&max[i][j]);
        }
    }
    int avail[3] = {3, 3, 2};
    int need[5][3];
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
    printf("Need Matrix:\n");
    for(i = 0; i < n; i++) {
        printf("P%d : ", i);
        for(j = 0; j < m; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
    int finish[5] = {0};
    int safeSeq[5];
    int count = 0;
    while(count < n) {
```

```

int found = 0;
for(i = 0; i < n; i++) {
    if(finish[i] == 0) {
        for(j = 0; j < m; j++) {
            if(need[i][j] > avail[j])
                break;
        }
        if(j == m) {
            for(k = 0; k < m; k++) {
                avail[k] += alloc[i][k];
            }
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}
if(found == 0) {
    printf("\nSystem is NOT in safe state\n");
    return 0;
}
printf("\nSystem is in SAFE state\n");
printf("\nSafe Sequence:\n");
for(i = 0; i < n; i++) {
    printf("P%d", safeSeq[i]);
    if(i != n - 1)
        printf(" -> ");
}
printf("\n");
return 0;
}

```

OUTPUT:

```
C:\Users\BMSCECSE\Desktop\ X + v
Enter n and m values: 5 3
Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Need Matrix:
P0 : 7 4 3
P1 : 1 2 2
P2 : 6 0 0
P3 : 0 1 1
P4 : 4 3 1
System is in SAFE state
Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0) execution time : 50.2
Press any key to continue.
printf("%d", need[i][j]),
```

a) Deadlock Detection

```
#include <stdio.h>

#define MAX 10 int

main() {
    int n, m;
    int Allocation[MAX][MAX]; int
    Request[MAX][MAX]; int
    Available[MAX];
    int Work[MAX]; int
    Finish[MAX]; int i, j,
    k;
    printf("Enter Number of Processes: "); scanf("%d",
    &n);
    printf("Enter Number of Resource Types: ");
    scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n"); for(i =
    0; i < n; i++) {
        for(j = 0; j < m; j++) { scanf("%d",
            &Allocation[i][j]);
        }
    }
    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &Request[i][j]);
        }
    }
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &Available[i]);
    }
}
```

```

for(i = 0; i < m; i++) Work[i] =
    Available[i];
for(i = 0; i < n; i++) { int
    allZero = 1;
    for(j = 0; j < m; j++) {
        if(Allocation[i][j] != 0) {
            allZero = 0;
            break;
        }
    }
    if(allZero) Finish[i]
        = 1;
    else
        Finish[i] = 0;
}
int found; do {
    found = 0;
    for(i = 0; i < n; i++) {
        if(Finish[i] == 0) {
            for(j = 0; j < m; j++) { if(Request[i][j]
                > Work[j])
                    break;
            }
            if(j == m) {
                for(k = 0; k < m; k++) Work[k] +=
                    Allocation[i][k];
                Finish[i] = 1;
                found = 1;
            }
        }
    }
}

```

```
    }  
} while(found); int  
deadlock = 0;  
printf("\nDeadlocked Processes:\n");  
for(i = 0; i < n; i++) {  
    if(Finish[i] == 0) {  
        printf("P%d\n", i);  
        deadlock = 1;  
    }  
}  
}  
if(deadlock == 0)  
    printf("No Deadlock Detected\n"); return  
0;  
}
```

OUTPUT:

```
C:\Users\BMSCECSE\Desktop\ X + v
Enter number of processes: 4
Enter number of resources: 3
Enter Allocation Matrix:
1 0 2
2 1 1
1 0 3
1 2 2
Enter Request Matrix:
0 0 1
1 0 2
0 0 0
3 3 0
Enter Available Resources:
0 0 0
Process P3 is in deadlock
Process returned 0 (0x0) execution time : 31.955 s
Press any key to continue.
|
```

```
C:\Users\BMSCECSE\Desktop\ X + v
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
Enter Available Resources:
0 0 0
No Deadlock Detected
Process returned 0 (0x0) execution time : 50.379 s
Press any key to continue.
|
```

LAB 6

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit

b) Best-fit

c) First-fit

```
#include <stdio.h>

void firstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i]; break;
            }
        }
    }
    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n"); for
    (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);

        if (allocation[i] != -1) printf("%d\n",
            allocation[i] + 1);
    }
}
```

else

```
printf("Not Allocated\n");
```

```

    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++)
    {
        int bestIdx = -1;
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }
    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n"); for
    (int i = 0; i < n; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]); if
        (allocation[i] != -1)

```

```

        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++)
    {
        int worstIdx = -1;
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1)
        {
            allocation[i] = worstIdx; blockSize[worstIdx]
            -= processSize[i];
        }
    }
    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n"); for
    (int i = 0; i < n; i++)

```

```

{
    printf("%d\t\t%d\t\t", i + 1, processSize[i]); if
    (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blockSize[m];
    int block1[m], block2[m], block3[m]; printf("Enter
    sizes of %d memory blocks:\n", m); for (int i = 0; i <
    m; i++)
    {
        scanf("%d", &blockSize[i]);
        block1[i] = blockSize[i];
        block2[i] = blockSize[i];
        block3[i] = blockSize[i];
    }
    printf("Enter number of processes: "); scanf("%d",
    &n);
    int processSize[n];
    printf("Enter sizes of %d processes:\n", n); for
    (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

```

```

firstFit(block1, m, processSize, n);
bestFit(block2, m, processSize, n);
worstFit(block3, m, processSize, n);
return 0;
}

```

OUTPUT:

```

"C:\Users\student\Desktop\1\  X + v
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.    Process Size    Block No.
1              212          2
2              417          5
3              112          2
4              426        Not Allocated

--- Best Fit ---
Process No.    Process Size    Block No.
1              212          4
2              417          2
3              112          3
4              426          5

--- Worst Fit ---
Process No.    Process Size    Block No.
1              212          5
2              417          2
3              112          5
4              426        Not Allocated

Process returned 0 (0x0)    execution time : 45.328 s
Press any key to continue.
|

```

LAB 7

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>

int frames[10];

void fifo(int pages[], int
n, int f) {
    int i, j, k = 0, fault =
0, found;
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++)
    {
        found = 0;
        for(j = 0; j < f;
j++) {
            if(frames[j] ==
pages[i]) {
                found = 1;
                break;}}
        if(!found) {
            frames[k] =
pages[i];
            k = (k + 1) % f;
            fault++;}}
    printf("FIFO Page
Faults = %d\n", fault);
}

void lru(int pages[], int
n, int f) {
```

```

    int i, j, least, pos,
    fault = 0;
    int time[10], counter
= 0;
    for(i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;}
    for(i = 0; i < n; i++)
    {
        int found = 0;
        for(j = 0; j < f;
j++) {
            if(frames[j] ==
pages[i]) {
                counter++;
                time[j] =
counter;
                found = 1;
                break;}}
        if(!found) {
            least = time[0];
            pos = 0;
            for(j = 0; j < f;
j++) {
                if(frames[j]
== -1) {
                    pos = j;
                    break;}
                if(time[j] <
least) {
                    least =
time[j];

```

```

        pos = j;}}
    frames[pos] =
pages[i];
    counter++;
    time[pos] =
counter;
    fault++;}}
    printf("LRU Page
Faults = %d\n", fault);
}
void optimal(int
pages[], int n, int f) {
    int i, j, k, pos,
farthest, fault = 0;
    for(i = 0; i < f; i++)
        frames[i] = -1;
    for(i = 0; i < n; i++)
    {
        int found = 0;
        for(j = 0; j < f;
j++) {
            if(frames[j] ==
pages[i]) {
                found = 1;
                break;}}
        if(!found) {
            pos = -1;
            for(j = 0; j < f;
j++) {
                if(frames[j]
== -1) {
                    pos = j;

```

```

        break;}}
    if(pos == -1) {
        farthest = i;
        pos = 0;
        for(j = 0; j <
f; j++) {
            int next =
-1;
            for(k = i +
1; k < n; k++) {

if(frames[j] ==
pages[k]) {
                next =
k;

break;}}

            if(next ==
-1) {
                pos = j;
                break;}
            if(next >
farthest) {
                farthest
= next;
                pos =
j;}}}}
        frames[pos] =
pages[i];
        fault++;}}
    printf("Optimal Page
Faults = %d\n", fault);

```

```
}  
  
int main() {  
    int pages[20], n, f, i;  
    printf("Enter number  
of pages: ");  
    scanf("%d", &n);  
    printf("Enter page  
reference string: ");  
    for(i = 0; i < n; i++)  
        scanf("%d",  
&pages[i]);  
    printf("Enter number  
of frames: ");  
    scanf("%d", &f);  
    fifo(pages, n, f);  
    lru(pages, n, f);  
    optimal(pages, n, f);  
    return 0;  
}
```

OUTPUT:

```
C:\Users\BMSCECSE\Desktop\ X + v
Enter number of pages: 12
Enter page reference string: 1 4 0 2 6 1 2 7 3 0 2 4
Enter number of frames: 3
FIFO Page Faults = 11
LRU Page Faults = 11
Optimal Page Faults = 9

Process returned 0 (0x0)   execution time : 19.554 s
Press any key to continue.
|
```